CONTENTS

10			Implement the following File Allocation Strategies using C programs			
	A		Sequential			
	B		Indexed			
	C		Linked			
11			Create and manage a Virtual Machine using VirtualBox			

Exp. No: 1	Installation of Operating system: Windows/ Linux
Date:	

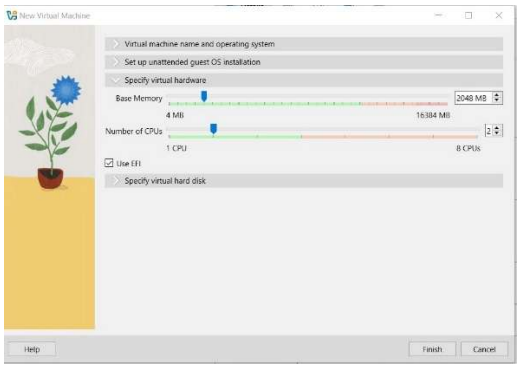
Aim:

To install an Operating System such as **Windows** or **Linux** on a computer system successfully.

Procedure:

For Windows Installation:

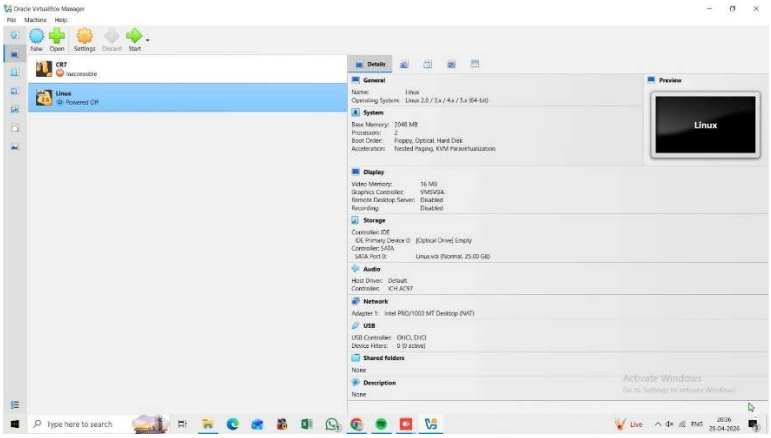
1. Download the Windows ISO file from the official website.
2. Create a bootable USB drive using Rufus or any bootable software.
3. Insert the USB drive into the computer.
4. Restart the system and enter BIOS/Boot Menu.
5. Select USB drive as the first boot device.
6. Save settings and restart.
7. Windows setup screen will appear.
8. Select language, time, and keyboard settings.
9. Click **Install Now**.
10. Enter product key (if required).
11. Select Custom Installation.



12. Choose the partition and click **Next**.
13. Wait for installation to complete.
14. Set username, password, and preferences.
15. Finish setup and open Windows desktop.

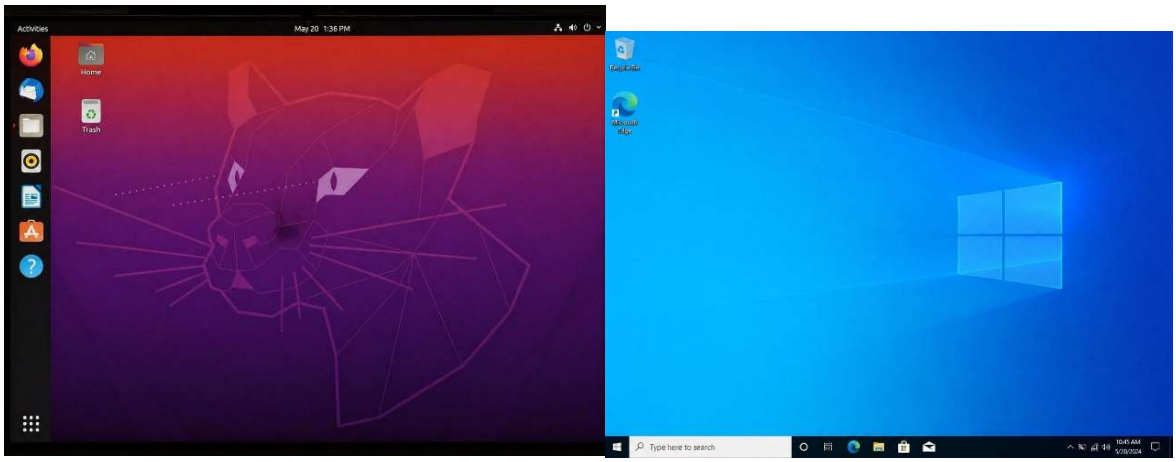
For Linux Installation:

1. Download Linux ISO file (Ubuntu / Fedora etc.).
2. Create a bootable USB drive using Rufus / Balena Etcher.
3. Insert USB drive into the computer.
4. Restart system and open BIOS/Boot Menu.
5. Select USB drive as boot device.
6. Linux installation menu will appear.
7. Select **Install Linux**.



8. Choose language and keyboard layout.
9. Select installation type.
10. Create partitions or choose erase disk option.
11. Click **Install Now**.
12. Set username, password, and timezone.
13. Wait for installation to finish.
14. Restart the computer and remove USB drive.
15. Linux desktop will open successfully.

Output:



Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result:

Thus, the Operating System (**Windows / Linux**) was installed successfully on the computer.

Exp. No: 2	Illustrate UNIX commands and Shell Programming
Date:	

Aim

To study and illustrate basic UNIX commands and develop simple shell programs.

Procedure

Basic UNIX Commands

1. Open the terminal in a UNIX/Linux system.
2. Execute the following commands:

Command	Description
pwd	Displays present working directory
ls	Lists files and directories
cd	Changes directory
mkdir	Creates a new directory
rmdir	Removes an empty directory
touch	Creates a new file
cat	Displays file contents
cp	Copies files
mv	Moves or renames files
rm	Deletes files

Shell Programming

Program : Addition of Two Numbers

```
echo "Enter two numbers:"
read a b
sum=`expr $a + $b`
echo "Sum is $sum"
```

Output:

Enter two numbers:

10 20

Sum is 30

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, basic UNIX commands were executed successfully and shell programs were written and verified with correct output.

Exp. No: 3	Process Management using System Calls: Fork, Exec
Date:	Getpid(), Exit, Wait, Close

Aim

To implement process management in Unix/Linux using system calls such as fork(), exec(), getpid(), wait(), and exit().

Procedure

1. Start the program execution.
2. Use fork() to create a child process.
3. Check the return value of fork():
    - If 0, it is the child process.
    - If positive, it is the parent process.
4. In the child process:
    - Display its Process ID using getpid().
    - Use exec() to execute a new program (ls command).
5. In the parent process:
    - Display its Process ID using getpid().
    - Use wait() to wait for the child process to complete.
6. After child execution, parent resumes.
7. Use exit() to terminate the process.
8. Stop the program.

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
   pid_t pid;
```

```
    pid = fork();
    if (pid < 0) {
        printf("Fork Failed\n");
        exit(1);
    }
    else if (pid == 0) {
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        execl("/bin/ls", "ls", NULL);
        printf("Exec failed\n");
        exit(0);
    }
    else {
        printf("\n--- Parent Process ---\n");
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("\nChild process completed\n");
    }
    return 0;
}
```

Output:

--- Parent Process ---

Parent PID: 2451

--- Child Process ---

Child PID: 2452

a.out

main.c

Child process completed.

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the process management system calls such as fork(), exec(), getpid(), wait(), and exit() were successfully implemented and executed.

Exp. No: 4	Write C programs to implement the various
Date:	CPU Scheduling Algorithms

Aim

To write a C program to implement various CPU Scheduling Algorithms such as:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round Robin (RR)

Procedure

1. Start the program.
2. Enter the number of processes.
3. Input burst time and (if needed) priority for each process.
4. Display menu to choose scheduling algorithm:
    - FCFS
    - SJF
    - Priority
    - Round Robin
5. Based on choice:
    - Perform scheduling calculation
    - Compute waiting time and turnaround time
6. Display results in tabular format.
7. Stop the program

Program:

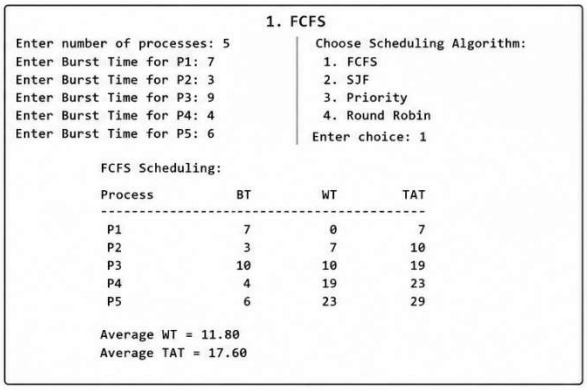
```
#include <stdio.h>

int main() {

   int n, i, j, choice, tq;

   int bt[20], wt[20], tat[20], p[20], pr[20];
```

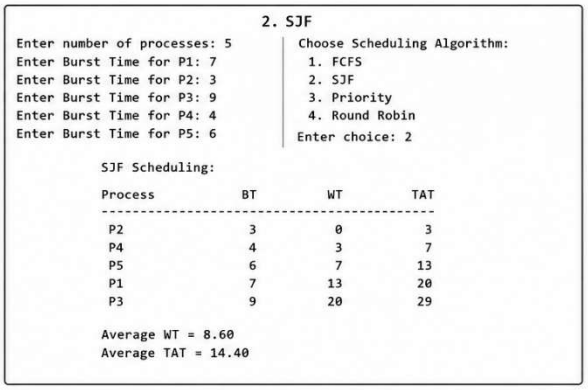
```
printf("Enter number of processes: ");
scanf("%d", &n);
for(i = 0; i < n; i++) {
   p[i] = i + 1;
   printf("Enter Burst Time for P%d: ", p[i]);
   scanf("%d", &bt[i]);
}
printf("\nChoose Scheduling Algorithm:");
printf("\n1. FCFS\n2. SJF\n3. Priority\n4. Round Robin\n");
printf("Enter choice: ");
scanf("%d", &choice);
if(choice == 1) {
   wt[0] = 0;
   for(i = 1; i < n; i++)
      wt[i] = wt[i-1] + bt[i-1];
   printf("\nFCFS Scheduling:\n");
}
else if(choice == 2) {
   for(i = 0; i < n-1; i++) {
      for(j = i+1; j < n; j++) {
         if(bt[i] > bt[j]) {
            int temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
         }
      }
   }
   wt[0] = 0;
   for(i = 1; i < n; i++)
      wt[i] = wt[i-1] + bt[i-1];
```

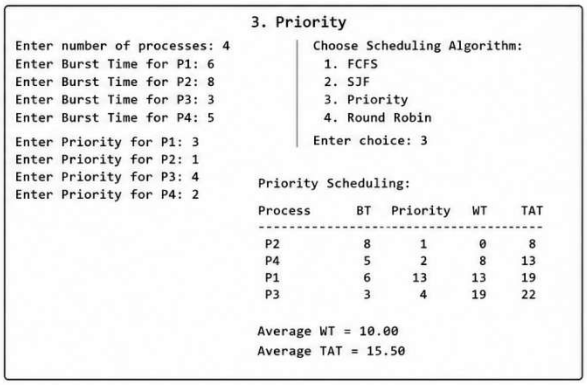
```
    printf("\nSJF Scheduling:\n");
}
else if(choice == 3) {
    for(i = 0; i < n; i++) {
        printf("Enter Priority for P%d: ", i+1);
        scanf("%d", &pr[i]);
    }
    for(i = 0; i < n-1; i++) {
        for(j = i+1; j < n; j++) {
            if(pr[i] > pr[j]) {
                int temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i-1] + bt[i-1];
    printf("\nPriority Scheduling:\n");
}
else if(choice == 4) {
    int rem[20], time = 0, done;
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    for(i = 0; i < n; i++)
```

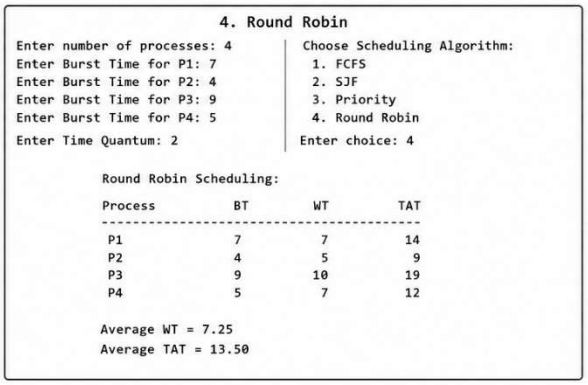
```
        rem[i] = bt[i];
      do {
        done = 1;
        for(i = 0; i < n; i++) {
          if(rem[i] > 0) {
            done = 0;
            if(rem[i] > tq) {
              time += tq;
              rem[i] -= tq;
            } else {
              time += rem[i];
              wt[i] = time - bt[i];
              rem[i] = 0;
            }
          }
        }
      } while(!done);
      printf("\nRound Robin Scheduling:\n");
    }
    for(i = 0; i < n; i++)
      tat[i] = bt[i] + wt[i];
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
      printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    }
    return 0;
}
```

Output:









Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the C program to implement various CPU Scheduling Algorithms such as FCFS, SJF, Priority, and Round Robin was successfully executed and the waiting time and turnaround time were calculated.

Exp. No: 5	Illustrate the inter process communication strategy
Date:	

Aim

To illustrate Inter Process Communication (IPC) between a parent and child process using a pipe in C.

Procedure

1. Start the program.
2. Declare an integer array fd[2] for pipe file descriptors.
3. Create a pipe using pipe(fd).
4. Use fork() to create a child process.
5. In the **parent process**:
    - Close the read end of the pipe (fd[0]).
    - Write a message into the pipe using write().
6. In the **child process**:
    - Close the write end of the pipe (fd[1]).
    - Read the message from the pipe using read().
7. Display the received message in the child process.
8. End the program.

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>

int main() {
   int fd[2];
   pid_t pid;
   char write_msg[] = "Hello from Parent";
```

```
    char read_msg[50];
    if (pipe(fd) == -1) {
        printf("Pipe failed\n");
        return 1;
    }
    pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    if (pid > 0) {
        close(fd[0]);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        printf("Parent sent message: %s\n", write_msg);
    }
    else {
        close(fd[1]);
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received message: %s\n", read_msg);
    }
    return 0;
}
```

Output:

Parent sent message: Hello from Rushi

Child received message: Hello from Rushi

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the inter process communication between parent and child processes was successfully implemented using a pipe system call in C.

Exp. No: 6	Implement mutual exclusion by Semaphores
Date:	

Aim

To implement mutual exclusion using semaphores for synchronizing processes and avoiding race conditions.

Procedure

1. Start the program.
2. Include required header files (stdio.h, pthread.h, semaphore.h, unistd.h).
3. Declare a semaphore variable.
4. Initialize the semaphore using sem_init() with value 1 (binary semaphore).
5. Create multiple threads using pthread_create().
6. In each thread:
    - Call sem_wait() to enter critical section.
    - Access and modify shared resource.
    - Call sem_post() to exit critical section.
7. Wait for all threads to finish using pthread_join().
8. Destroy the semaphore using sem_destroy().
9. Stop the program.

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
sem_t mutex;
int shared = 0;
void* process(void* arg) {
    sem_wait(&mutex);

    int x = shared;
    printf("Thread %ld is reading shared value: %d\n", (long)arg, x);
    x++;
    sleep(1);
    shared = x;
    printf("Thread %ld updated shared value to: %d\n", (long)arg, shared);
    sem_post(&mutex);
    return NULL;
}
int main() {
    pthread_t t1, t2;
    sem_init(&mutex, 0, 1);
    pthread_create(&t1, NULL, process, (void*)1);
    pthread_create(&t2, NULL, process, (void*)2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&mutex);
    printf("Final value of shared variable: %d\n", shared);
    return 0;
}
```

Output:

Thread 1 is reading shared value: 1

Thread 1 updated shared value to: 2

Thread 2 is reading shared value: 2

Thread 2 updated shared value to: 3

Final value of shared variable: 3

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, mutual exclusion was successfully implemented using semaphores, ensuring that only one thread accesses the critical section at a time and preventing race conditions.

Exp. No: 7	Write a C program to Implement Deadlock Detection Algorithm and Avoid Deadlock using Banker's Algorithm
Date:	

Aim

To write a C program to:

1. Detect deadlock in a system.
2. Avoid deadlock using Banker's Algorithm.

Procedure

Deadlock Detection

1. Start the program.
2. Input number of processes and resources.
3. Enter Allocation matrix.
4. Enter Request matrix.
5. Enter Available resources.
6. Initialize all processes as unfinished.
7. Check if any process can complete with available resources.
8. If yes → release its resources and mark it finished.
9. Repeat until no further processes can proceed.
10. If any process remains unfinished → **Deadlock detected**.

Banker's Algorithm (Deadlock Avoidance)

1. Input number of processes and resources.
2. Enter Allocation matrix.
3. Enter Maximum matrix.
4. Calculate **Need = Max – Allocation**.
5. Enter Available resources.
6. Find a process whose Need ≤ Available.
7. Allocate resources → update Available.
8. Mark process as completed.
9. Repeat until all processes finish.

Coding:

```
#include <stdio.h>
int main() {
   int n, m, i, j, k;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resources: ");
   scanf("%d", &m);
   int alloc[10][10], max[10][10], need[10][10];
   int avail[10], finish[10] = {0}, safeSeq[10];
   printf("\nEnter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         scanf("%d", &alloc[i][j]);
   printf("\nEnter Max Matrix:\n");
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         scanf("%d", &max[i][j]);
   printf("\nEnter Available Resources:\n");
   for(i = 0; i < m; i++)
      scanf("%d", &avail[i]);
   for(i = 0; i < n; i++)
      for(j = 0; j < m; j++)
         need[i][j] = max[i][j] - alloc[i][j];
   int count = 0;
   while(count < n) {
      int found = 0;
      for(i = 0; i < n; i++) {
         if(finish[i] == 0) {
            int flag = 1;
```

```
            for(j = 0; j < m; j++) {
                if(need[i][j] > avail[j]) {
                    flag = 0;
                    break;
                }
            }
            if(flag) {
                for(k = 0; k < m; k++)
                    avail[k] += alloc[i][k];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0) {
        printf("\nSystem is NOT in safe state (Deadlock possible)\n");
        return 0;
    }
  }
  printf("\nSystem is in SAFE state\nSafe sequence: ");
  for(i = 0; i < n; i++)
    printf("P%d ", safeSeq[i]);
  return 0;
}
```

Output:

Enter number of processes: 3

Enter number of resources: 3

Enter Allocation Matrix

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Available Resources

3 3 2

System is in SAFE state

Safe sequence: P1 P0 P2

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the deadlock detection and deadlock avoidance using Banker's Algorithm were successfully implemented, and the system was verified to be in a safe or unsafe state.

Exp. No: 8	Write C programs to implement the following Memory Allocation Methods
Date:	

Aim

To write a C program to implement different memory allocation strategies:

- First Fit
- Best Fit
- Worst Fit

Procedure

1. Start the program.
2. Enter number of memory blocks and their sizes.
3. Enter number of processes and their memory requirements.
4. Choose allocation method:
    - First Fit → allocate first sufficient block
    - Best Fit → allocate smallest sufficient block
    - Worst Fit → allocate largest available block
5. Allocate memory based on selected method.
6. Display allocation result.
7. Stop the program.

Coding:

```
#include <stdio.h>
void firstFit(int blocks[], int m, int process[], int n) {
   int allocation[20];
   for (int i = 0; i < n; i++) allocation[i] = -1;
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (blocks[j] >= process[i]) {
            allocation[i] = j;
```

```
            blocks[j] -= process[i];
            break;
        }
    }
}
printf("\nFirst Fit Allocation:\n");
for (int i = 0; i < n; i++)
    printf("Process %d (%d) -> Block %d\n", i+1, process[i], allocation[i]+1);
}
void bestFit(int blocks[], int m, int process[], int n) {
int allocation[20];
for (int i = 0; i < n; i++) allocation[i] = -1;
for (int i = 0; i < n; i++) {
    int best = -1;
    for (int j = 0; j < m; j++) {
        if (blocks[j] >= process[i]) {
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
        }
    }
    if (best != -1) {
        allocation[i] = best;
        blocks[best] -= process[i];
    }
}
printf("\nBest Fit Allocation:\n");
for (int i = 0; i < n; i++)
    printf("Process %d (%d) -> Block %d\n", i+1, process[i], allocation[i]+1);
}
```

```
void worstFit(int blocks[], int m, int process[], int n) {
    int allocation[20];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= process[i]) {
                if (worst == -1 || blocks[j] > blocks[worst])
                    worst = j;
            }
        }
        if (worst != -1) {
            allocation[i] = worst;
            blocks[worst] -= process[i];
        }
    }
    printf("\nWorst Fit Allocation:\n");
    for (int i = 0; i < n; i++)
        printf("Process %d (%d) -> Block %d\n", i+1, process[i], allocation[i]+1);
}
int main() {
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[20];
    printf("Enter sizes of blocks:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);
```

```
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int process[20];
    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &process[i]);
    int choice;
    printf("\n1. First Fit\n2. Best Fit\n3. Worst Fit\nEnter choice: ");
    scanf("%d", &choice);
    int temp[20];
    for (int i = 0; i < m; i++) temp[i] = blocks[i];
    if (choice == 1)
        firstFit(temp, m, process, n);
    else if (choice == 2)
        bestFit(temp, m, process, n);
    else if (choice == 3)
        worstFit(temp, m, process, n);
    else
        printf("Invalid choice");
    return 0;
}
```

Output:

```
Enter number of memory blocks: 6
Enter sizes of blocks:
150  250  350  450  550  650
Enter number of processes: 5
Enter sizes of processes:
115  275  530  160  430

1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 1  (First Fit Algorithm)

First Fit Allocation:
Process 1 (115) -> Block 1
Process 2 (275) -> Block 3
Process 3 (530) -> Block 6
Process 4 (160) -> Block 4
Process 5 (430) -> Block 5
----------------------------
Remaining Block Sizes:
Block 1: 35
Block 2: 250
Block 3: 75
Block 4: 290
Block 5: 120
Block 6: 120

Best Fit Allocation:
Process 1 (115) -> Block 1
Process 2 (275) -> Block 2
Process 3 (530) -> Block 6
Process 4 (160) -> Block 3
Process 5 (430) -> Block 4
----------------------------
Remaining Block Sizes:
Block 1: 35
Block 2: 0
Block 3: 190
Block 4: 290
Block 5: 120
Block 6: 120

Worst Fit Allocation:
Process 1 (115) -> Block 6
Process 2 (275) -> Block 5
Process 3 (530) -> Block -1
Process 4 (160) -> Block 4
Process 5 (430) -> Block 3
----------------------------
Remaining Block Sizes:
Block 1: 150
Block 2: 250
Block 3: 120
Block 4: 290
Block 5: 120
Block 6: 535
```

Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the C program to implement memory allocation methods such as First Fit, Best Fit, and Worst Fit was successfully executed and memory was allocated according to the selected strategy.

Exp. No: 9	Write C programs to implement the various Page
Date:	Replacement Algorithms

Aim

To write a C program to implement various page replacement algorithms:

- FIFO (First In First Out)
- LRU (Least Recently Used)
- OPT (Optimal Page Replacement)

Procedure

1. Start the program.
2. Enter the number of pages in the reference string.
3. Enter the page reference string.
4. Enter the number of frames.
5. Choose the algorithm:
    - FIFO → replace oldest page
    - LRU → replace least recently used page
    - OPT → replace page not used for longest future time
6. Simulate page replacement.
7. Count and display page faults.
8. Stop the program.

Coding:

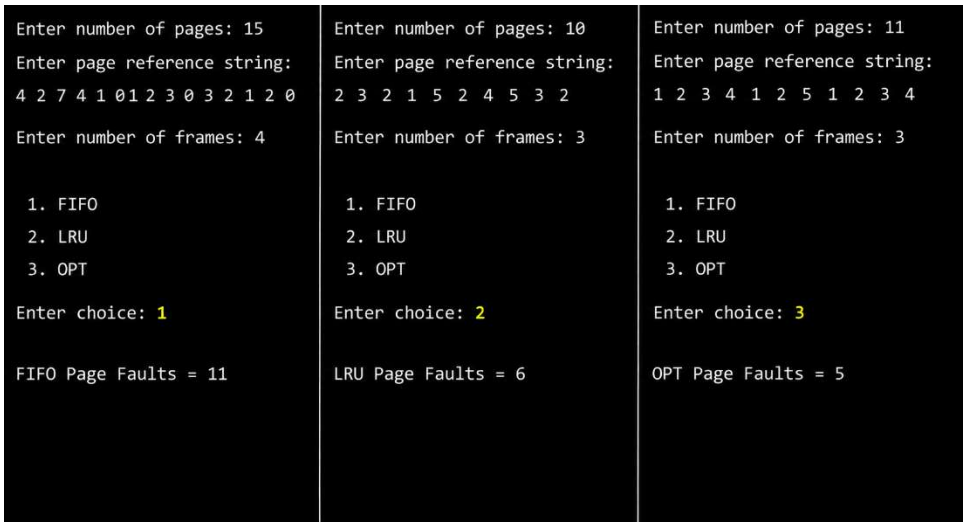
```
#include <stdio.h>
int main() {
   int pages[50], frames[10], temp[10];
   int n, f, i, j, k, pos, faults = 0;
   int choice;
   printf("Enter number of pages: ");
   scanf("%d", &n);
```

```
printf("Enter page reference string:\n");
for(i = 0; i < n; i++)
   scanf("%d", &pages[i]);
printf("Enter number of frames: ");
scanf("%d", &f);
printf("\n1. FIFO\n2. LRU\n3. OPT\nEnter choice: ");
scanf("%d", &choice);
for(i = 0; i < f; i++)
   frames[i] = -1;
if(choice == 1) {
   int index = 0;
   for(i = 0; i < n; i++) {
      int found = 0;
      for(j = 0; j < f; j++) {
         if(frames[j] == pages[i]) {
            found = 1;
            break;
         }
      }
      if(!found) {
         frames[index] = pages[i];
         index = (index + 1) % f;
         faults++;
      }
   }
   printf("\nFIFO Page Faults = %d\n", faults);)
else if(choice == 2) {
   int time[10], counter = 0;
   for(i = 0; i < n; i++) {
      int found = 0;
      for(j = 0; j < f; j++) {
```

```
            if(frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(!found) {
            int min = 0;
            for(j = 1; j < f; j++)
                if(time[j] < time[min])
                    min = j;
            frames[min] = pages[i];
            counter++;
            time[min] = counter;
            faults++;
        }
    }
    printf("\nLRU Page Faults = %d\n", faults);
}
else if(choice == 3) {
    for(i = 0; i < n; i++) {
        int found = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if(!found) {
```

```
            int farthest = i, index = -1;
            for(j = 0; j < f; j++) {
               int k;
               for(k = i+1; k < n; k++) {
                  if(frames[j] == pages[k])
                     break;
               }
               if(k == n) {
                  index = j;
                  break;
               }
               if(k > farthest) {
                  farthest = k;
                  index = j;
               }
            }
            if(index == -1)
               index = 0;
            frames[index] = pages[i];
            faults++;
         }
      }
      printf("\nOPT Page Faults = %d\n", faults);
   }
   else {
      printf("Invalid choice\n");
   }
   return 0;
}
```

Output:



Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the C program to implement page replacement algorithms such as FIFO, LRU, and Optimal was successfully executed and the number of page faults was determined.

Exp. No: 10	
Date:	**Implement the following File Allocation Strategies using C programs**

Aim

To write a C program to implement different file allocation strategies:

- Sequential Allocation
- Indexed Allocation
- Linked Allocation

Procedure

1. Start the program.
2. Display menu with choices:
    1. Sequential Allocation
    2. Indexed Allocation
    3. Linked Allocation
    4. Exit
3. Read user choice.
4. Based on choice:

    Perform corresponding allocation method.
5. Display allocation result.
6. Repeat until user selects Exit.
7. Stop the program.

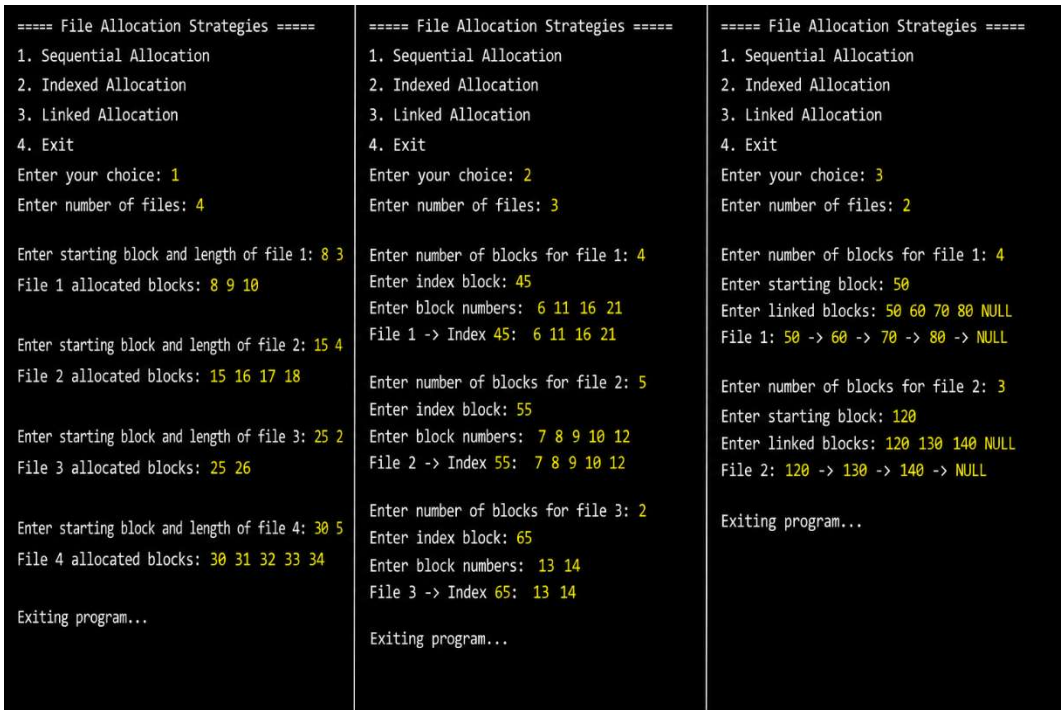
Coding:

```
#include <stdio.h>
int main() {
   int choice;
   while (1) {
      printf("\n===== File Allocation Strategies =====\n");
      printf("1. Sequential Allocation\n");
```

```
printf("2. Indexed Allocation\n");
printf("3. Linked Allocation\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
if (choice == 1) {
    int n, i, j, start, len;
    printf("\nEnter number of files: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nEnter starting block and length of file %d: ", i+1);
        scanf("%d %d", &start, &len);
        printf("File %d allocated blocks: ", i+1);
        for (j = 0; j < len; j++)
            printf("%d ", start + j);
        printf("\n");
    }
}
else if (choice == 2) {
    int n, i, j, index, blocks[20], count;
    printf("\nEnter number of files: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nEnter number of blocks for file %d: ", i+1);
        scanf("%d", &count);
        printf("Enter index block: ");
        scanf("%d", &index);
        printf("Enter block numbers: ");
        for (j = 0; j < count; j++)
            scanf("%d", &blocks[j]);
        printf("File %d -> Index %d: ", i+1, index);
```

```
            for (j = 0; j < count; j++)
                printf("%d ", blocks[j]);
            printf("\n");
        }
    }
    else if (choice == 3) {
        int n, i, j, start, blocks[20], count;
        printf("\nEnter number of files: ");
        scanf("%d", &n);
        for (i = 0; i < n; i++) {
            printf("\nEnter number of blocks for file %d: ", i+1);
            scanf("%d", &count);
            printf("Enter starting block: ");
            scanf("%d", &start);
            printf("Enter linked blocks: ");
            for (j = 0; j < count; j++)
                scanf("%d", &blocks[j]);
            printf("File %d: %d -> ", i+1, start);
            for (j = 0; j < count; j++)
                printf("%d -> ", blocks[j]);
            printf("NULL\n");
        }
    }
    else if (choice == 4) {
        printf("\nExiting program...\n");
        break;}
    else {
        printf("\nInvalid choice! Try again.\n");}
  }
  return 0;
}
```

Output:



Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, the menu-driven C program for implementing Sequential, Indexed, and Linked file allocation strategies was successfully executed.

Exp. No: 11	Create and manage a Virtual Machine using VirtualBox
Date:	

Aim

To create and manage a Virtual Machine using Oracle VM VirtualBox.

Procedure

Part A: Install VirtualBox

1. Download Oracle VM VirtualBox from the official website.
2. Run the installer and follow the setup instructions.
3. Launch VirtualBox after installation.

Part B: Create a Virtual Machine

1. Open VirtualBox and click **New**.
2. Enter:
    - Name of VM (e.g., Ubuntu)
    - Type: Linux / Windows
    - Version: Based on OS
3. Allocate **RAM** (e.g., 2GB or more).
4. Create a **Virtual Hard Disk**:
    - Type: VDI (VirtualBox Disk Image)
    - Allocation: Dynamically allocated
    - Size: 20GB or more
5. Click **Create**.

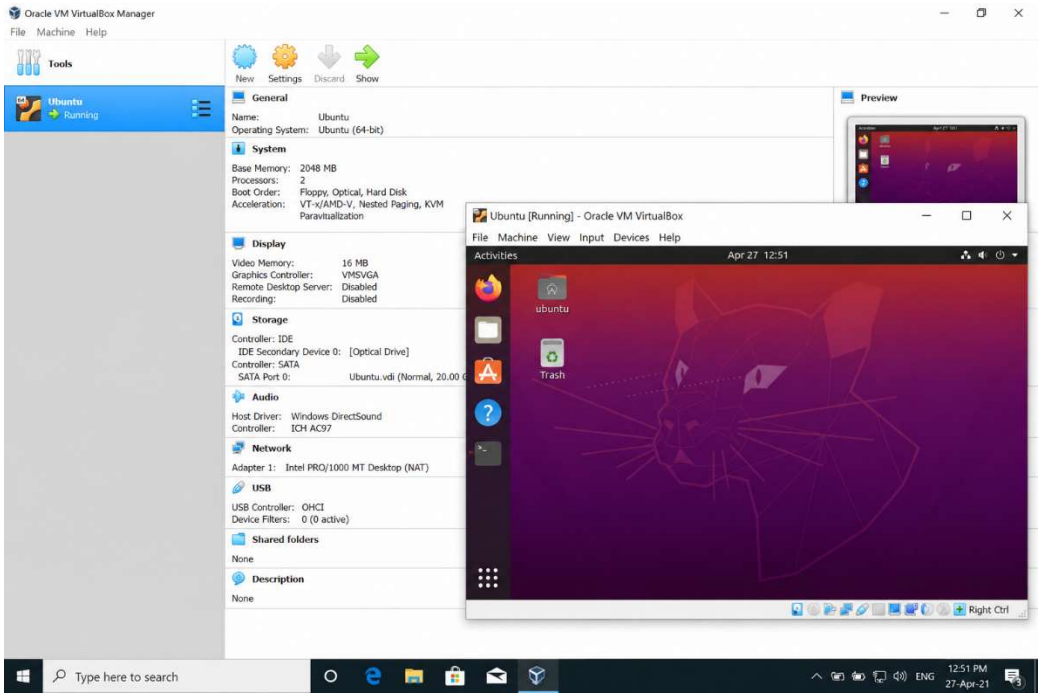
Part C: Install Operating System

1. Select the created VM → Click **Start**.
2. Choose OS ISO file (e.g., Ubuntu ISO).
3. Follow on-screen installation steps:
    - Language selection
    - Disk partition
    - User creation
4. Complete installation and restart VM.

Part D: Manage the Virtual Machine

1. Start/Stop VM using **Start / Power Off**.
2. Adjust settings:
    - RAM / CPU cores
    - Display / Storage
3. Take **Snapshots** to save system state.
4. Install **Guest Additions** for better performance:
    - Improves graphics, mouse integration, clipboard sharing

Output:



Department of CSE (AIML)		
Criteria	**Max. Marks**	**Marks Obtained**
Procedure/ Algorithm, Program	**50**	
Output	**20**	
Timeline and Report	**20**	
Viva	**10**	
Total	**100**	

Result

Thus, a Virtual Machine was successfully created and managed using Oracle VM VirtualBox. The operating system was installed and executed inside the virtual environment.